ESP32 OV2640 摄像头 Web 界面 - 简化版代码解析

文件信息

- **原始文件**: index_ov2640.txt (52,474 字符)
- **简化文件**: index_ov2640_simplified.html (37,928 字符)
- **精简比例**: 27.7%
- **删除内容**: Advanced Settings 区域（Register Get/Set、CLK、Window）

基础操作信息链路传输逻辑

1. 整体架构

```
┌──────────────────┐      HTTP GET      ┌──────────────────┐
│   浏览器前端      │ ◄────────────────► │   ESP32 服务器    │
│  (HTML/JS/CSS)   │                    │  (C++/Arduino)   │
└──────────────────┘                    └──────────────────┘

      │                                        │
      │  1. 获取初始状态: GET /status          │
      │  2. 更新参数:     GET /control?var=x&val=y │
      │  3. 视频流:       GET :81/stream       │
      │  4. 拍照:         GET /capture         │
```

2. 核心通信流程详解

2.1 页面初始化流程

```javascript
// 页面加载完成后执行
document.addEventListener('DOMContentLoaded', function (event) {
  var baseHost = document.location.origin  // 获取服务器地址，如 http://192.168.1.34
  var streamUrl = baseHost + ':81'         // 视频流端口为 81

  // 步骤 1: 读取相机初始状态
  fetch(`${baseHost}/status`)
    .then(response => response.json())     // 解析 JSON 响应
    .then(state => {
      // 步骤 2: 更新所有 UI 控件到初始值
      document.querySelectorAll('.default-action').forEach(el => {
        updateValue(el, state[el.id], false)  // false = 不发送到服务器
      })
    })
})
```

服务器响应示例 (`/status`):

```
{
  "framesize": 8,       // 分辨率 VGA
  "quality": 10,        // JPEG 品质
  "brightness": 0,      // 亮度
  "contrast": 0,        // 对比度
  "awb": 1,             // 白平衡开启
```

```
  "aec": 1,            // 自动曝光开启
  // ... 其他参数
}
```

2.2 参数更新流程 当用户操作任何带有 `class="default-action"` 的控件时：

```
// 用户操作触发流程：
// 1. 用户拖动滑块/点击开关/选择下拉框
// 2. 触发 onchange 事件
// 3. 调用 updateConfig(el) 发送数据到服务器

document.querySelectorAll('.default-action').forEach(el => {
  el.onchange = () => updateConfig(el)
})

function updateConfig(el) {
  // 根据控件类型获取值
  let value
  switch (el.type) {
    case 'checkbox':
      value = el.checked ? 1 : 0    // 开关：true/false → 1/0
      break
    case 'range':
    case 'select-one':
      value = el.value               // 滑块/选择框：直接取数值
      break
    case 'button':
    case 'submit':
      value = '1'                    // 按钮：固定值 1
      break
    default:
      return
  }

  // 构建请求 URL：/control?var= 参数名 &val= 参数值
  const query = `${baseHost}/control?var=${el.id}&val=${value}`

  // 发送 GET 请求到 ESP32
  fetch(query)
    .then(response => {
      console.log(`请求完成，状态码：${response.status}`)
    })
}
```

实际请求示例：

- 修改分辨率：`GET http://192.168.1.34/control?var=framesize&val=8`
- 调整亮度：`GET http://192.168.1.34/control?var=brightness&val=1`
- 关闭白平衡：`GET http://192.168.1.34/control?var=awb&val=0`

Kimi.ai 生成

2.3 视频流传输流程

```
const streamUrl = baseHost + ':81'  // 视频流使用独立端口

// 开始流传输
const startStream = () => {
  view.src = `${streamUrl}/stream`    // 直接设置 img 标签的 src
  show(viewContainer)
  streamButton.innerHTML = '停止图像传输'
}

// 停止流传输
const stopStream = () => {
  window.stop()                       // 中止所有网络请求
  streamButton.innerHTML = '实时图像数据流传输'
}
```

特点:

- 使用 MJPEG (Motion JPEG) 格式，本质是连续发送的 JPEG 图片流
- 直接赋值给 `<img>` 标签的 `src`，浏览器自动处理流式数据
- 独立端口 81 避免与 HTTP 控制命令 (端口 80) 冲突

2.4 拍照流程

```
stillButton.onclick = () => {
  stopStream()  // 先停止视频流

  // 添加时间戳防止缓存
  view.src = `${baseHost}/capture?_cb=${Date.now()}`
  show(viewContainer)
}
```

原理:

- `Date.now()` 生成唯一时间戳，确保每次请求都是新的图片
- ESP32 返回单张 JPEG 图像数据

2.5 图片保存流程

```
saveButton.onclick = () => {
  // 创建临时 canvas
  var canvas = document.createElement("canvas")
  canvas.width = view.width
  canvas.height = view.height

  // 将当前图像绘制到 canvas
  var context = canvas.getContext('2d')
  context.drawImage(view, 0, 0)

  // 转换为 JPEG DataURL
  var dataURL = canvas.toDataURL('image/jpeg')
```

```
  // 设置下载链接
  saveButton.href = dataURL
  saveButton.download = "YYYYMMDDhhmmss.jpg"  // 自动生成文件名
}
```

---

3. 核心数据结构

3.1 支持的参数列表（28 个 default-action 元素）

参数名	类型	范围/选项	说明
framesize	select	0-13	分辨率（96x96 到 UXGA）
quality	range	4-63	JPEG 压缩品质
brightness	range	-2 到 2	亮度调节
contrast	range	-2 到 2	对比度调节
saturation	range	-2 到 2	饱和度调节
special_effect	select	0-6	特效（负片/黑白/红通道等）
zoom-scale	range	1-4	数字变焦
awb	checkbox	0/1	自动白平衡
awb_gain	checkbox	0/1	白平衡增益
wb_mode	select	0-4	白平衡模式（晴天/多云/室内等）
aec	checkbox	0/1	自动曝光控制
aec2	checkbox	0/1	自动曝光 DSP
ae_level	range	-2 到 2	曝光等级
aec_value	range	0-1200	手动曝光值
agc	checkbox	0/1	自动增益控制
agc_gain	range	0-30	手动增益值
gainceiling	range	0-6	最大增益上限
bpc	checkbox	0/1	死点校正
wpc	checkbox	0/1	白点校正
raw_gma	checkbox	0/1	原始伽马校正
lenc	checkbox	0/1	镜头畸变校正
hmirror	checkbox	0/1	水平镜像
vflip	checkbox	0/1	垂直翻转
dcw	checkbox	0/1	降采样/分辨率调整
colorbar	checkbox	0/1	色彩测试条
led_intensity	range	0-255	LED 补光灯亮度
face_detect	checkbox	0/1	人脸检测
face_recognize	checkbox	0/1	人脸识别

3.2 特殊联动逻辑

```
// 1. 自动曝光 vs 手动曝光
```

```
// 当 AEC 开启时，隐藏手动曝光值调节滑块
aec.onchange = () => {
  updateConfig(aec)
  aec.checked ? hide(exposure) : show(exposure)
}

// 2. 自动增益 vs 手动增益
// 当 AGC 开启时，显示增益上限，隐藏手动增益
agc.onchange = () => {
  updateConfig(agc)
  if (agc.checked) {
    show(gainCeiling)
    hide(agcGain)
  } else {
    hide(gainCeiling)
    show(agcGain)
  }
}

// 3. 白平衡增益 vs 白平衡模式
// 只有当 AWB Gain 开启时，才显示 WB Mode 选择
awb.onchange = () => {
  updateConfig(awb)
  awb.checked ? show(wb) : hide(wb)
}

// 4. 分辨率 vs 人脸检测
// 分辨率高于 CIF (value > 5) 时，强制关闭人脸检测
framesize.onchange = () => {
  updateConfig(framesize)
  if (framesize.value > 5) {
    updateValue(detect, false)
    updateValue(recognize, false)
  }
}
```

---

4. 错误处理机制

```
function fetchUrl(url, cb){
  fetch(url)
    .then(function (response) {
      if (response.status !== 200) {
        // HTTP 错误 (404, 500 等)
        cb(response.status, response.statusText)
      } else {
        response.text()
          .then(data => cb(200, data))
          .catch(err => cb(-1, err))  // 数据解析错误
      }
```

```
    })
    .catch(function (err) {
      cb(-1, err)  // 网络连接错误
    })
}
```

错误类型:

- 外层 `catch`: 网络不通/DNS 失败/服务器宕机/CORS 错误
- 状态码判断: HTTP 协议错误 (404/500/403 等)
- 内层 `catch`: 数据格式错误/解析失败

---

5. 与 ESP32 服务器的交互协议

5.1 ESP32 端 (C++/Arduino)

```
// 在 app_httpd.cpp 中注册 URI 处理函数
httpd_uri_t status_uri = {
    .uri       = "/status",
    .method    = HTTP_GET,
    .handler   = status_handler,    // 返回 JSON 状态
    .user_ctx  = NULL
};

httpd_uri_t control_uri = {
    .uri       = "/control",
    .method    = HTTP_GET,
    .handler   = cmd_handler,       // 处理参数更新
    .user_ctx  = NULL
};

httpd_uri_t capture_uri = {
    .uri       = "/capture",
    .method    = HTTP_GET,
    .handler   = capture_handler,   // 返回 JPEG 图像
    .user_ctx  = NULL
};

httpd_uri_t stream_uri = {
    .uri       = "/stream",
    .method    = HTTP_GET,
    .handler   = stream_handler,    // 返回 MJPEG 流
    .user_ctx  = NULL
};
```

5.2 通信流程图

```
浏览器                                  ESP32
  |                                       |
```

```
├─ GET /status ──────────────────────►|
|                                     |
|◄─ {"framesize":8,"quality":10,...}──┤
|                                     |
|  [用户拖动亮度滑块到 1]               |
|                                     |
├─ GET /control?var=brightness&val=1 ─►|
|                                     |
|◄─ 200 OK (空响应)────────────────────┤
|                                     |
|  [用户点击" 实时图像数据流传输"]        |
|                                     |
├─ GET :81/stream (保持连接) ──────────►|
|◄─ MJPEG 数据流 (multipart/x-mixed-replace)
|  --boundary
                    |
|  Content-Type: image/jpeg
      |
|  Content-Length: 1234
         |
|
                                |
|  [JPEG 数据]                          |
|  --boundary
                    |
|  [下一帧...]                          |
```

---

6. 关键设计模式

6.1 单向数据流

- **初始化**: Server → Client (获取状态)
- **更新**: Client → Server (用户操作)
- **无双向绑定**: 服务器不会主动推送状态变化

6.2 简单的 REST-like API

- 全部使用 GET 请求（简化实现）
- 查询参数传递数据 (`?var=x&val=y`)
- 无请求体，无复杂头部

6.3 直接 DOM 操作

- 无虚拟 DOM 框架
- 直接操作 `el.value`, `el.checked`
- 手动管理控件联动逻辑

---

7. 代码简化对比

功能	原始代码	简化后代码
HTML 行数	~800	~550
JavaScript 函数	15+	8
事件绑定	复杂 (reg-action + default-action)	简单 (仅 default-action)
网络请求函数	setReg/getReg/setXclk/setWindow/fetchUrl	仅 fetchUrl
控件类型	基础 + 高级寄存器	仅基础控件
学习曲线	陡峭	平缓

总结

简化后的代码保留了 ESP32 OV2640 摄像头的**核心功能**:

1. ✅ 28 个基础相机参数的设置
2. ✅ 视频流实时预览
3. ✅ 单帧拍照
4. ✅ 图片保存到本地
5. ✅ 人脸检测/识别控制

删除的高级功能 (不影响基础使用):

- ❌ 直接寄存器读写 (Register Get/Set)
- ❌ 时钟频率调整 (CLK)
- ❌ 自定义窗口裁剪 (Window)

这套简化代码更适合**学习物联网 Web 控制的基础原理**，避免了被复杂的寄存器操作和高级设置分散注意力。